



دانشگاه اصفهان
دانشکده ریاضی و آمار

عنوان درس: هوش مصنوعی

گزارش جامع و تحلیل فنی پروژه شماره ۳

جستجوی خصمانه در بازی اتلو با Minimax و Alpha-Beta

دکتر فریا نصیری مفخم

فؤاد رضایی آدریانی، محسن محمود آبادی

نیمسال دوم ۱۴۰۴-۱۴۰۵

<https://github.com/MohsenMahmoodAbadi/AI26-PRJ3>

استاد درس:

دانشجویان:

نیمسال تحصیلی:

مخزن گیت‌هاب:

۱ مقدمه و صورت مسئله

در این پروژه هدف طراحی عامل تصمیم‌گیر برای بازی اتلو بود. بازی روی صفحه مربعی اجرا می‌شود و در نسخه اولیه پروژه اندازه صفحه برابر 6×6 است؛ با این حال پیاده‌سازی عامل‌ها به اندازه صفحه وابسته نشده و از مقدار `game.size` استفاده می‌کند تا در محیط‌های ارزیابی بزرگ‌تر نیز قابل اجرا باشد. هر بازیکن در نوبت خود باید حرکتی انجام دهد که دست‌کم یک مهره حریف بین مهره جدید و یکی از مهره‌های خودش محصور شود. مهره‌های محصور شده برگردانده می‌شوند و بازی زمانی تمام می‌شود که هیچ‌کدام از دو بازیکن حرکت قانونی نداشته باشند. در این پروژه دو عامل اصلی تکمیل شد: `MinimaxAgent` و `AlphaBetaAgent`. هر دو عامل از یک تابع ارزیابی مشترک استفاده می‌کنند، اما عامل دوم با هرس آلفا-بتا تعداد گره‌های بررسی شده را کم می‌کند. علاوه بر این، فایل `main.py` به گونه‌ای کامل شد که عامل‌ها را در عمق‌های مختلف در برابر عامل تصادفی و عامل حریصانه اجرا کند و آمار برد، اختلاف مهره، تعداد گره و زمان میانگین را گزارش دهد.

۲ تابع ارزیابی

از آنجا که در بیشتر حالت‌ها امکان جستجوی کامل تا انتهای بازی وجود ندارد، عامل‌ها در عمق محدود از تابع ارزیابی استفاده می‌کنند. تابع ارزیابی پیاده‌سازی شده فقط اختلاف تعداد مهره‌ها را در نظر نمی‌گیرد؛ زیرا در اتلو، به‌ویژه در ابتدای بازی، داشتن مهره بیشتر همیشه به معنی موقعیت بهتر نیست. معیارهای زیر در امتیازدهی استفاده شده‌اند:

- **اختلاف مهره‌ها:** در انتهای بازی معیار اصلی برد است، اما در ابتدای بازی وزن کمتری دارد.
 - **تحرك يا Mobility:** اختلاف تعداد حرکت‌های قانونی بازیکن و حریف. هرچه بازیکن انتخاب‌های بیشتری داشته باشد، احتمال کنترل بازی بیشتر می‌شود.
 - **گوشه‌ها:** گوشه‌های صفحه پایدارترین خانه‌ها هستند و گرفتن آن‌ها امتیاز زیادی دارد.
 - **ریسک خانه‌های کنار گوشه خالی:** خانه‌های مجاور گوشه خالی ممکن است به حریف فرصت گرفتن گوشه بدهند، بنابراین جریمه می‌شوند.
 - **لبه‌ها و وزن مکانی:** خانه‌های لبه معمولاً از خانه‌های داخلی امن‌تر هستند و برای هر خانه وزن مکانی جداگانه در نظر گرفته شده است.
 - **مهره‌های مرزی:** مهره‌ای که کنار خانه خالی قرار دارد آسیب‌پذیرتر است، پس تعداد زیاد مهره‌های مرزی امتیاز منفی ایجاد می‌کند.
- فرمول کلی تابع ارزیابی را می‌توان به صورت زیر خلاصه کرد:

$$\begin{aligned} Eval(s, p) = & w_d \Delta Disc + w_m \Delta Mobility + 4 \times Positional \\ & + 25 \times Corner - 12 \times CornerRisk \\ & + 5 \times Edge - 4 \times Frontier \end{aligned} \quad (1)$$

وزن اختلاف مهره و تحرك با مرحله بازی تغییر می‌کند. در نیمه اول بازی تحرك مهم‌تر است، اما در نیمه پایانی اختلاف مهره‌ها اهمیت بیشتری پیدا می‌کند. اگر وضعیت نهایی باشد، تابع ارزیابی مقدار بسیار بزرگ مثبت یا منفی برمی‌گرداند تا برد و باخت نهایی بر هر تخمین میانی غالب شود.

```
1 if game.game_over():
2     if disc_diff > 0:
3         return 100000 + disc_diff
4     if disc_diff < 0:
5         return -100000 + disc_diff
6     return 0
7
8 return (
9     disc_weight * disc_diff
10    + mobility_weight * mobility
11    + 4 * position_score
12    + 25 * corner_score
13    - 12 * corner_risk
14    + 5 * edge_score
15    - 4 * frontier_score
16 )
```

Listing 1: خلاصه منطق ارزیابی

۳ عامل Minimax

عامل `MinimaxAgent` در هر نوبت درخت بازی را تا عمق مشخص جستجو می‌کند. در سطح‌های مربوط به بازیکن اصلی، بیشینه مقدار انتخاب می‌شود و در سطح‌های مربوط به حریف، کمینه مقدار انتخاب می‌شود. مقدار برگ‌ها با تابع ارزیابی محاسبه می‌شود. اگر یک بازیکن در وضعیت میانی حرکت قانونی نداشته باشد اما بازی تمام نشده باشد، نوبت به حریف داده می‌شود و عمق یک واحد کاهش پیدا می‌کند تا حالت پاس نیز به درستی مدل شود.

```

1 if depth == 0 or game.game_over():
2     return evaluate(game, root_player), None
3
4 current_player = root_player if maximizing else -root_player
5 moves = game.get_valid_moves(current_player)
6
7 if not moves:
8     return minimax(game, depth - 1, not maximizing, root_player)
9
10 for move in moves:
11     child = game.copy()
12     child.make_move(current_player, *move)
13     value, _ = minimax(child, depth - 1, not maximizing, root_player)

```

Listing 2: Minimax اصلی منطق

پیچیدگی زمانی این روش در حالت کلی نمایی است. اگر میانگین تعداد حرکت‌های ممکن را با b و عمق را با d نشان دهیم، تعداد گره‌های بالقوه از مرتبه $O(b^d)$ است. به همین دلیل افزایش عمق از ۳ به ۴ در آزمایش‌ها باعث افزایش محسوس زمان اجرای Minimax شد.

۴ عامل Alpha-Beta

عامل AlphaBetaAgent همان تصمیم‌گیری Minimax را انجام می‌دهد، اما دو کران α و β را نگه می‌دارد. α بهترین مقدار قطعی پیداشده برای بیشینه‌ساز و β بهترین مقدار قطعی پیداشده برای کمینه‌ساز است. هرگاه مشخص شود ادامه بررسی یک شاخه نمی‌تواند تصمیم نهایی را بهتر کند، آن شاخه هرس می‌شود.

```

1 alpha = max(alpha, best_value)
2 if beta <= alpha:
3     break

```

Listing 3: Alpha-Beta در هرس شرط

برای افزایش اثر هرس، حرکت‌ها قبل از گسترش با یک ارزیابی سریع مرتب شدند؛ در سطح بیشینه‌ساز حرکت‌های امیدبخش‌تر زودتر بررسی می‌شوند و در سطح کمینه‌ساز حرکت‌های ضعیف‌تر برای بازیکن اصلی جلوتر قرار می‌گیرند. این مرتب‌سازی مقدار Minimax را تغییر نمی‌دهد، اما در حالت‌هایی که چند حرکت ارزش نهایی برابر دارند ممکن است انتخاب ظاهری عامل با Minimax متفاوت شود. در عوض، تعداد گره‌های لازم برای رسیدن به همان مقدار به شکل محسوسی کاهش می‌یابد.

۵ روش آزمایش

برای هر ترکیب عامل، عمق و حریف، ۲۰ بازی اجرا شد. برای کاهش اثر برتری رنگ شروع‌کننده، عامل مورد ارزیابی در بازی‌های زوج با مهره سیاه و در بازی‌های فرد با مهره سفید بازی کرد. حریف‌ها عبارت بودند از:

- RandomAgent که از بین حرکت‌های قانونی به صورت تصادفی انتخاب می‌کند.
 - GreedyAgent که حرکتی را انتخاب می‌کند که در همان نوبت بیشترین افزایش تعداد مهره را ایجاد کند.
- در هر بازی، تعداد برد، باخت، مساوی، میانگین اختلاف مهره نهایی، میانگین تعداد گره بررسی‌شده در هر تصمیم و میانگین زمان هر تصمیم ثبت شد. برای تکرارپذیر بودن نتایج، در ابتدای هر بازی مقدار بذر تصادفی ثابت تنظیم شد.

۶ نتایج آزمایش‌ها

جدول زیر خروجی اجرای main.py را نشان می‌دهد. ستون AvgDiff میانگین اختلاف مهره عامل اصلی نسبت به حریف است؛ مقدار مثبت یعنی عامل اصلی با فاصله بیشتری برده است.

Agent	Opponent	Depth	Games	Wins	Draws	Losses	WinRate	AvgDiff
Minimax	Random	2	20	19	0	1	95.0%	18.50
Minimax	Greedy	2	20	20	0	0	100.0%	19.00
Minimax	Random	3	20	19	0	1	95.0%	19.15
Minimax	Greedy	3	20	20	0	0	100.0%	20.00
Minimax	Random	4	20	20	0	0	100.0%	23.95
Minimax	Greedy	4	20	20	0	0	100.0%	26.00
AlphaBeta	Random	2	20	19	0	1	95.0%	18.00
AlphaBeta	Greedy	2	20	20	0	0	100.0%	19.00
AlphaBeta	Random	3	20	19	0	1	95.0%	19.25
AlphaBeta	Greedy	3	20	20	0	0	100.0%	20.00
AlphaBeta	Random	4	20	20	0	0	100.0%	23.00
AlphaBeta	Greedy	4	20	20	0	0	100.0%	17.00

نتایج نشان می‌دهد که هر دو عامل در برابر عامل‌های پایه عملکرد بسیار خوبی دارند. در بیشتر حالت‌ها عمق ۴ بهترین اختلاف مهره را ایجاد کرده است. تنها موردی که اختلاف میانگین Alpha-Beta کمتر از Minimax شده، بازی عمق ۴ در برابر Greedy است. علت اصلی این است که Alpha-Beta به دلیل مرتب‌سازی حرکت‌ها در حالت‌های هم‌ارزش ممکن است حرکت متفاوتی انتخاب کند؛ با این حال همچنان در تمام ۲۰ بازی برنده شده است.

۷ مقایسه Minimax و Alpha-Beta

جدول بعدی هزینه محاسباتی دو روش را مقایسه می‌کند. کاهش تعداد گره‌ها در عمق‌های بالاتر بسیار واضح است.

Opponent	Depth	MM Nodes	AB Nodes	Node Reduction	MM Time	AB Time	Time Reduction
Random	2	35.1	17.1	51.3%	5.75	8.19	-42.4%
Greedy	2	29.6	15.2	48.6%	4.91	7.21	-46.8%
Random	3	227.9	62.2	72.7%	38.29	26.22	31.5%
Greedy	3	219.5	62.9	71.3%	35.48	25.13	29.2%
Random	4	1418.1	139.8	90.1%	232.15	84.03	63.8%
Greedy	4	1186.9	142.9	88.0%	181.69	76.46	57.9%

در عمق ۲ با وجود کاهش تعداد گره‌ها، زمان Alpha-Beta بیشتر شده است؛ زیرا هزینه مرتب‌سازی حرکت‌ها و ارزیابی اولیه آن‌ها از سود هرس بیشتر بوده است. از عمق ۳ به بعد، تعداد شاخه‌ها بیشتر می‌شود و هرس آلفا-بتا اثر خود را نشان می‌دهد. در عمق ۴، Alpha-Beta در برابر Random حدود ۹۰ درصد گره کمتر بررسی کرده و زمان تصمیم‌گیری را حدود ۶۴ درصد کاهش داده است.

۸ تحلیل پرسش‌های پروژه

۱.۸ اثر افزایش عمق جستجو

افزایش عمق به عامل اجازه می‌دهد پیامدهای بلندمدت‌تر یک حرکت را ببیند. در عمق ۲ عامل بیشتر به سود کوتاه‌مدت و تابع ارزیابی نزدیک است، اما در عمق ۴ می‌تواند دام‌هایی مانند دادن گوشه به حریف را بهتر تشخیص دهد. در نتایج نیز افزایش عمق معمولاً اختلاف مهره نهایی را بهتر کرده است. هزینه این بهبود، رشد نمایی تعداد گره‌ها و افزایش زمان اجراست.

۲.۸ اثر Alpha-Beta بر زمان اجرا

آلفا-بتا مقدار نظری Minimax را تغییر نمی‌دهد، اما شاخه‌هایی را حذف می‌کند که در تصمیم نهایی اثری ندارند. در عمق‌های کم، سربار مرتب‌سازی می‌تواند زمان را بیشتر کند؛ ولی در عمق‌های بالاتر، کاهش تعداد گره‌ها غالب می‌شود. نتیجه عملی این است که Alpha-Beta برای عمق‌های بزرگ‌تر انتخاب مناسب‌تری از Minimax خام است.

۳.۸ معیارهای استفاده‌شده در تابع ارزیابی

تابع ارزیابی از ترکیب اختلاف مهره، تحرک، گوشه‌ها، ریسک خانه‌های مجاور گوشه، وزن مکانی، لبه‌ها و مهره‌های مرزی استفاده می‌کند. این معیارها تلاش می‌کنند بین سه هدف تعادل ایجاد کنند: برد نهایی با مهره بیشتر، کنترل آینده بازی با حرکت‌های بیشتر، و جلوگیری از دادن موقعیت‌های پایدار به حریف.

۴.۸ مهم‌ترین عامل موفقیت

مهم‌ترین عامل موفقیت ترکیب تحرک و کنترل گوشه‌ها بود. عامل حریصانه فقط افزایش فوری مهره را می‌بیند، اما عامل پیاده‌سازی‌شده حاضر است در کوتاه‌مدت مهره کمتری داشته باشد تا حرکت‌های آینده بیشتری حفظ کند و گوشه‌ها را از دست ندهد. همین تفاوت باعث شد در برابر Greedy نرخ برد ۱۰۰ درصد به دست آید.

۵.۸ نقاط ضعف تابع ارزیابی

تابع ارزیابی هنوز یک تخمین است و پایداری واقعی همه مهره‌ها را محاسبه نمی‌کند. همچنین مرتب‌سازی Alpha-Beta بر اساس ارزیابی تک‌مرحله‌ای انجام می‌شود و ممکن است در موقعیت‌های دارای تساوی نهایی، مسیر بازی متفاوتی نسبت به Minimax خام ایجاد کند. ضعف دیگر این است که وزن‌ها با آزمایش دستی انتخاب شده‌اند و با روش‌های یادگیری یا تنظیم خودکار بهینه نشده‌اند.

۹ جمع‌بندی

در این پروژه دو عامل کامل برای بازی اتلو پیاده‌سازی شد. عامل Minimax درخت بازی را تا عمق محدود بررسی می‌کند و عامل Alpha-Beta همان تصمیم‌گیری را با هرس شاخه‌های بی‌اثر سریع‌تر انجام می‌دهد. تابع ارزیابی طراحی‌شده فقط بر تعداد مهره‌ها تکیه ندارد و معیارهای مهم اتلو مانند تحرک، گوشه‌ها، ریسک کنار گوشه، لبه‌ها و مهره‌های مرزی را در نظر می‌گیرد.

آزمایش‌های انجام‌شده نشان دادند که عامل‌های پیاده‌سازی‌شده در برابر Greedy و Random بسیار قوی عمل می‌کنند. افزایش عمق معمولاً کیفیت بازی را بهتر کرد و Alpha-Beta در عمق‌های بالاتر کاهش بزرگی در تعداد گره‌های بررسی‌شده و زمان اجرا ایجاد کرد. بنابراین برای ارزیابی مسابقه‌ای، استفاده از Alpha-Beta با عمق مناسب و تابع ارزیابی چندمعیاره، گزینه‌ای کارآمدتر از Minimax خام است.